Tarea programada

Sistema de Gestión de Productos

Curso: Programación II
Carrera: Ingeniería en Sistemas de Información
Valor: 15% de la nota final del curso
Modalidad: Individual o en pareja
Defensa: Obligatoria

1. Descripción de la tarea

Como parte de la evaluación del curso de Programación 2, se deberá desarrollar una aplicación de escritorio en C# utilizando Windows Forms, que permita administrar los productos de una pequeña tienda.

El sistema se denominará: **Sistema de Gestión de Productos**

La aplicación deberá permitir administrar dos elementos principales:

- **Categorías**
- **Productos**

La información deberá almacenarse en una base de datos SQL Server y deberá existir una relación entre ambas tablas.

La finalidad de la actividad es que el estudiante integre los conocimientos estudiados durante el curso en una aplicación pequeña pero funcional, aplicando programación orientada a objetos, programación por capas, validaciones, operaciones CRUD y conexión con una base de datos.

2. Objetivo general

Desarrollar una aplicación de escritorio en C# y Windows Forms que permita gestionar categorías y productos mediante operaciones de registro, consulta, modificación y eliminación, utilizando programación orientada a objetos, programación por capas y una base de datos relacional en SQL Server.

3. Objetivos específicos

Al finalizar la tarea, el estudiante deberá ser capaz de:

1. Diseñar una aplicación utilizando formularios de Windows Forms.
2. Crear y utilizar clases y objetos en C#.
3. Aplicar propiedades, métodos y encapsulamiento.
4. Separar la interfaz de usuario de la lógica del programa.
5. Implementar validaciones de información.
6. Conectar una aplicación C# con SQL Server.
7. Realizar operaciones de inserción, consulta, modificación y eliminación.
8. Utilizar controles gráficos para facilitar la interacción con el usuario.
9. Aplicar estructuras condicionales y otros elementos de programación vistos durante el curso.
10. Integrar una solución funcional utilizando una base de datos relacional.

4. Descripción del sistema

La aplicación deberá permitir administrar los productos de una tienda.

Cada producto pertenecerá a una categoría.

Por ejemplo:

Categorías

- Computación
- Accesorios
- Impresión

- Redes

Productos

- Teclado
- Mouse
- Monitor
- Impresora
- Cable de red
- Memoria USB

Los ejemplos anteriores son únicamente ilustrativos. El estudiante podrá utilizar otros nombres de categorías y productos, siempre que respete la estructura solicitada.

5. Base de datos

La base de datos deberá contener como máximo dos tablas.

Tabla 1: Categorias

Deberá contener como mínimo:

Campo	Tipo sugerido	Descripción
IdCategoria	INT	Identificador único
Nombre	VARCHAR	Nombre de la categoría
Descripcion	VARCHAR	Descripción de la categoría

La clave primaria será:

IdCategoria

Tabla 2: Productos

Deberá contener como mínimo:

Campo	Tipo sugerido	Descripción
IdProducto	INT	Identificador único
Nombre	VARCHAR	Nombre del producto
Precio	DECIMAL	Precio del producto
Stock	INT	Cantidad disponible
FechaRegistro	DATETIME	Fecha de registro
IdCategoria	INT	Categoría a la que pertenece

La clave primaria será:

IdProducto

La tabla Productos deberá tener una llave foránea:

IdCategoria → Categorias.IdCategoria

6. Requisitos funcionales

La aplicación deberá cumplir como mínimo con los siguientes requisitos.

RF-01. Administración de categorías

El sistema deberá permitir:

- Registrar una categoría.
- Consultar las categorías existentes.
- Modificar una categoría.
- Eliminar una categoría.

Antes de eliminar una categoría, el sistema deberá solicitar confirmación al usuario.

RF-02. Administración de productos

El sistema deberá permitir:

- Registrar un producto.
- Consultar los productos registrados.
- Modificar un producto.
- Eliminar un producto.

Antes de eliminar un producto, el sistema deberá solicitar confirmación.

RF-03. Asignación de categoría

Al registrar o modificar un producto, el usuario deberá seleccionar la categoría correspondiente.

No se deberá permitir registrar un producto sin categoría.

La selección podrá realizarse mediante un control apropiado, como ComboBox o ListBox.

RF-04. Búsqueda de productos

El sistema deberá incluir una opción para buscar productos.

Como mínimo deberá poder buscarse mediante el nombre del producto.

RF-05. Consulta mediante DataGridView

Los productos deberán mostrarse utilizando un DataGridView.

Como mínimo deberá visualizarse:

- Identificador.
- Nombre.
- Precio.
- Stock.

- Fecha de registro.
- Categoría.

RF-06. Cálculo del valor del inventario

El sistema deberá mostrar un valor calculado para cada producto:

Valor del inventario = Precio × Stock

Este valor deberá ser calculado por la aplicación y no almacenarse como un campo adicional en la base de datos.

RF-07. Fecha de registro

Al registrar un nuevo producto, la aplicación deberá establecer automáticamente la fecha de registro utilizando un mecanismo como:

DateTime.Now

RF-08. Validaciones

El sistema deberá validar como mínimo:

- Campos obligatorios.
- Precio mayor que cero.
- Stock igual o mayor que cero.
- Datos numéricos válidos.
- Selección obligatoria de categoría.

Se deberá utilizar ErrorProvider o un mecanismo equivalente para informar los errores de validación.

RF-09. Mensajes al usuario

La aplicación deberá mostrar mensajes mediante MessageBox para:

- Confirmar eliminaciones.

- Informar operaciones exitosas.
- Mostrar errores.
- Informar cuando no se encuentren resultados.

7. Requisitos de programación orientada a objetos

El proyecto deberá utilizar programación orientada a objetos.

Como mínimo deberá incluir:

Clase Categoría

Deberá representar la información de una categoría mediante propiedades y métodos cuando sean necesarios.

Clase Producto

Deberá representar la información de un producto mediante propiedades y métodos.

Además, el estudiante deberá demostrar el uso de:

- Clases.
- Objetos.
- Métodos.
- Propiedades.
- get y set.
- Encapsulamiento.

Se deberá evitar colocar toda la lógica del sistema directamente dentro de los eventos de los botones.

8. Programación por capas

El proyecto deberá estar organizado utilizando como mínimo:

Capa de Interfaz

Responsable de:

- Formularios.
- Controles.
- Entrada de datos.
- Presentación de información.

Capa de Lógica

Responsable de:

- Reglas del sistema.
- Validaciones.
- Procesamiento de información.
- Operaciones relacionadas con los objetos.

La organización de las clases deberá permitir identificar claramente las responsabilidades de cada parte del programa.

9. Requisitos técnicos

La tarea deberá desarrollarse utilizando:

- **Lenguaje:** C#.
- **Interfaz:** Windows Forms.
- **Entorno:** Visual Studio.
- **Base de datos:** Microsoft SQL Server.
- **Administración de la base de datos:** SQL Server Management Studio (SSMS).

No es necesario utilizar frameworks o tecnologías adicionales.

La aplicación deberá conectarse directamente a la base de datos y permitir realizar las operaciones solicitadas.

10. Controles mínimos esperados

Durante el desarrollo deberán utilizarse apropiadamente algunos de los controles estudiados durante el curso.

Como mínimo se deberá utilizar:

- Label.
- TextBox.
- Button.
- DataGridView.
- ComboBox o ListBox.
- MessageBox.
- ErrorProvider.

El estudiante podrá utilizar otros controles cuando sean necesarios para mejorar el sistema.

11. Calidad del código

El código deberá cumplir con condiciones básicas de calidad.

Se deberá:

- Utilizar nombres descriptivos para clases, variables, métodos y controles.
- Mantener una indentación consistente.
- Evitar código duplicado.
- Crear métodos para procesos que puedan reutilizarse.
- Separar correctamente la interfaz y la lógica.

- Evitar colocar todo el código en los eventos de los botones.
- Mantener una estructura organizada de carpetas y clases.

Ejemplos de nombres apropiados:

Producto

Categoria

GuardarProducto()

BuscarProducto()

txtNombre

txtPrecio

btnGuardar

dgvProductos

Se deberá evitar utilizar nombres genéricos como:

a

x

button1

textBox2

cuando pueda utilizarse un nombre descriptivo.

12. Manejo de errores

El programa deberá controlar situaciones previsibles, por ejemplo:

- Ingreso de datos incorrectos.
- Campos vacíos.
- Precio o stock inválidos.

- Problemas de conexión con la base de datos.
- Intento de realizar operaciones no permitidas.

Los errores deberán informarse al usuario mediante mensajes claros y comprensibles.

13. Git y GitHub

Se recomienda que el estudiante utilice Git y GitHub durante el desarrollo.

Se recomienda especialmente que el repositorio sea público, para facilitar la revisión del código.

El repositorio deberá contener, cuando se utilice:

- Código fuente.
- Script SQL.
- README.
- Documentación.

Se recomienda realizar commits durante el proceso de desarrollo y no únicamente al finalizar la tarea.

14. Documentación

La entrega deberá incluir como mínimo un archivo **README.md** con:

- Nombre de la tarea.
- Nombre del estudiante o integrantes.
- Descripción del sistema.
- Tecnologías utilizadas.
- Requisitos para ejecutar el programa.
- Instrucciones para crear la base de datos.

- Instrucciones para ejecutar el sistema.
- Información necesaria para la conexión a SQL Server.
- Descripción breve de las principales funcionalidades.

En caso de utilizar fuentes externas para elaborar la documentación, deberán ser citadas de acuerdo con APA 7.ª edición.

15. Uso de inteligencia artificial generativa

Se permite utilizar herramientas de inteligencia artificial generativa como apoyo para el desarrollo de la tarea.

Se podrá utilizar IA, por ejemplo, para:

- Consultar dudas sobre C#.
- Comprender mensajes de error.
- Solicitar explicaciones de conceptos.
- Obtener ejemplos de sintaxis.
- Apoyar la depuración.
- Revisar la redacción de la documentación.

No se permitirá utilizar la inteligencia artificial para generar completamente la tarea y presentarla como si hubiera sido desarrollada y comprendida por el estudiante.

El estudiante deberá ser capaz de explicar el funcionamiento del código que entregue.

Cuando se utilice IA de manera significativa, deberá indicarse en el README:

- Herramienta utilizada.
- Propósito de su utilización.
- Parte del proyecto en la que fue utilizada.

16. Defensa obligatoria

La defensa será obligatoria y tendrá como finalidad comprobar el funcionamiento del sistema y el dominio de los conceptos utilizados.

Durante la defensa, el estudiante deberá realizar una demostración funcional del programa y responder preguntas formuladas por el docente.

La demostración deberá incluir como mínimo:

1. Registro de una categoría.
2. Registro de un producto.
3. Consulta de información.
4. Búsqueda de un producto.
5. Modificación de información.
6. Eliminación de información.
7. Validación de datos.
8. Visualización del valor calculado del inventario.
9. Evidencia de la conexión con SQL Server.

El docente podrá solicitar que el estudiante explique fragmentos específicos del código, las clases utilizadas, la programación por capas o las operaciones realizadas sobre la base de datos.

Duración

La defensa tendrá una duración aproximada de:

- **Individual:** 8 a 10 minutos.
- **Pareja:** 10 a 15 minutos.

Consecuencia por no presentarse

La defensa es obligatoria.

En caso de que el estudiante no realice la defensa, la calificación de la tarea será de 1/15, salvo las situaciones debidamente justificadas de acuerdo con la normativa institucional.

Cuando la tarea sea realizada en pareja, ambos integrantes deberán participar en la defensa.

17. Modalidad de trabajo

La tarea podrá realizarse:

- Individualmente; o
- En pareja.

El máximo permitido será de dos estudiantes.

En caso de realizarla en pareja, ambos integrantes deberán participar activamente en el desarrollo.

El docente podrá realizar preguntas individuales durante la defensa para comprobar el dominio de cada integrante.

La pertenencia a una pareja no implica necesariamente que ambos estudiantes deban recibir la misma calificación.

18. Entregables

Cada estudiante o pareja deberá entregar:

1. Aplicación

- Proyecto completo de Visual Studio.
- Código fuente.
- Formularios y clases.
- Sistema funcional.

2. Base de datos

- Script .sql.
- Dos tablas como máximo.
- Llaves primarias.
- Llave foránea.
- Datos de prueba.

3. Documentación

- README.md.
- Instrucciones de instalación y ejecución.
- Declaración del uso de IA, cuando corresponda.

4. Repositorio

- Enlace al repositorio GitHub, preferiblemente público.

5. Defensa

- Demostración funcional obligatoria.

Rúbrica de evaluación

Criterio	**Puntos**
1. Base de datos y modelado	1.5
2. Interfaz gráfica	1.0
3. Programación orientada a objetos	2.0
4. Programación por capas	1.5
5. CRUD de categorías	1.5
6. CRUD de productos	2.0
7. Validaciones y manejo de errores	1.0
8. Conexión C# – SQL Server	1.5
9. Calidad y buenas prácticas del código	0.5
10. Documentación y Git/GitHub	0.5
11. Defensa y dominio del proyecto	1.0
TOTAL	**15.0**